Annexe — Programmes automate (STEP 7 SCL)

Projet de régulation séchoir et four B12 · Version définitive V1 du 19/10/2026. À compiler dans les projets STEP 7 (S7-SCL), à charger d'abord en mode observation (sorties calculées et archivées, commande existante conservée). Valeurs de départ à recaler par les essais du dossier.

Bloc	Automate	Rôle	Fichier
FB101 REGUL_SECHOIR	S7-300	bloc maître du séchoir	FB101_REGUL_SECHOIR.scl
FB102 CADENCE	S7-300	cadence et cycle réel	FB102_CADENCE.scl
FB103 LIMITES	S7-300	sondes, limites T°C1 / T°C4, registre arrière	FB103_LIMITES.scl
FB104 PLANIF_RECUP	S7-300	récupération, régimes, fréquences de base	FB104_PLANIF_RECUP.scl
FB105 BALANCELLE_BLOCAGE	S7-300	balancelle au blocage	FB105_BALANCELLE_BLOCAGE.scl
FB106 REGUL_HRA	S7-300	humidité avant recyclage	FB106_REGUL_HRA.scl
FB107 ASC_PILOTE	S7-300	reprise, plafond T°C4, zéro gaz	FB107_ASC_PILOTE.scl
FB114 PRESSION_PORTES	S7-412	pression et portes du four	FB114_PRESSION_PORTES.scl
FB115 CENTRALE_MOD	S7-412	modulation air/gaz des centrales	FB115_CENTRALE_MOD.scl
FB117 COMPRESSEUR_VAR	S7-300/400	compresseur à pression constante	FB117_COMPRESSEUR_VAR.scl

FB101 REGUL_SECHOIR (S7-300) — bloc maître du séchoir

```
// ===================================================================
// FB101 REGUL_SECHOIR - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Bloc maître du séchoir : modes de marche, boucle de l'humidité du
// recyclage (T_HR), sélecteur de la consigne ASC, ventilateurs, B1/B2,
// registre arrière, demande de récupération au four, alarmes.
// Projet définitif V1 du 19/10/2026. Valeurs de départ, à recaler par les essais.
//
// Ordre d'appel OB35 (100 ms) :
//   FB102 CADENCE -> FB103 LIMITES -> FB104 PLANIF_RECUP -> FB106 REGUL_HRA
//   -> FB107 ASC_PILOTE -> FB101 REGUL_SECHOIR (ce bloc)
// Sélecteur :
//   T_dem  = MAX(T_HR ; T_HRa)
//   T_cons = MIN(T_dem ; T_L1 ; T_L4 ; T_L7 ; T_max)  -> rampe montée 3 °C/min
//   Gel    = T_cons < T_dem - 0,5 °C  (renvoyé à FB106 et à la boucle T_HR)
// Observation (Auto_ASC / Auto_VENT = FAUX) : tout est calculé et archivé,
// les sorties suivent les valeurs réelles (reprise sans à-coup) et la
// commande existante reste en place.
// La gestion de flamme reste au calculateur du brûleur et aux coffrets.
// ===================================================================
FUNCTION_BLOCK FB101
VAR_INPUT
  dt         : REAL := 0.1;    // s
  Auto_ASC   : BOOL;           // consigne ASC, B1/B2 et registre pilotés par ce bloc
  Auto_VENT  : BOOL;           // fréquences des variateurs pilotées par ce bloc
  // ---- état de la ligne
  Mouleuse   : BOOL;           // mouleuse en marche
  Chaine     : BOOL;           // chaîne des balancelles en mouvement (FB102.Marche)
  // ---- mesures séchoir
  HR_rec     : REAL;           // %, humidité zone recyclage
  HR_rec_ok  : BOOL;
  HRa_f      : REAL;           // %, humidité avant recyclage filtrée (DB106.HRa_f)
  HRa_ok     : BOOL;
  T_sortie   : REAL;           // °C, sortie cheminée (capteur C1, à poser)
  Sortie_ok  : BOOL;           // FAUX tant que C1 n'est pas posé
  HR_sortie  : REAL;           // %, sortie cheminée
  Flamme     : BOOL;           // flamme ASC présente (coffret)
  ASC_act    : REAL;           // °C, consigne ASC réellement appliquée (suivi)
  F_AC_act, F_REC_act, F_EXT_act, F_BR_act : REAL;   // Hz réels des variateurs (suivi)
  // ---- FB103 LIMITES
  T_L1, T_L4 : REAL;           // °C
  M2_pos     : REAL;           // %
  Stop_crit  : BOOL;
  Def_sonde  : BOOL;
  // ---- FB104 PLANIF_RECUP
  F_AC_b, F_REC_b, F_EXT_b, F_BR_b : REAL;            // Hz de base
  Req_rec_b  : REAL;           // %, demande de récupération au four
  B2_antic   : BOOL;
  B1_antic   : BOOL;
  M2_plus    : REAL;           // %
  // ---- FB106 REGUL_HRA
  T_HRa      : REAL;           // °C
  dF_AC      : REAL;           // Hz
  // ---- FB107 ASC_PILOTE
  T_L7       : REAL;           // °C
  Stop_ASC   : BOOL;
  // ---- réglages (DB101, modifiables depuis WinCC)
  O1         : BOOL := FALSE;  // anticipation B2/B1 (FB104), après accord de l'exploitant (E10)
  t_standby  : REAL := 900.0;  // s d'arrêt mouleuse + chaîne avant le standby (15 min)
  t_rep_vent : REAL := 600.0;  // s, rampe des ventilateurs à la reprise
  t_rep_asc  : REAL := 120.0;  // s, ASC autorisé 2 min après la reprise
  SP_HR      : REAL := 30.0;   // %, centre de la plage recette (27-33 %) ; 36,5 après validation de l'essai 34-39 %
  DB_HR      : REAL := 3.0;    // %, demi-plage sans action
  Kp_HR      : REAL := 2.0;    // °C par point d'humidité du recyclage
  Ti_HR      : REAL := 600.0;  // s
  T0_HR      : REAL := 120.0;  // °C, point de départ de la boucle T_HR
  T_min      : REAL := 110.0;  // °C
  T_max      : REAL := 135.0;  // °C
  V_up       : REAL := 3.0;    // °C/min, montée maxi de la consigne (descente libre)
  T_repli    : REAL := 110.0;  // °C, consigne si une sonde de limite est en défaut
  AC_min     : REAL := 36.0;   // Hz, air chaud en production
  AC_max     : REAL := 45.0;
  F_min      : REAL := 25.0;
  F_max      : REAL := 50.0;
  HRa_L4     : REAL := 13.5;   // %, au-dessus : recyclage +5 Hz (sécurité produit)
  HR_ext_frc : REAL := 33.0;   // %, humidité recyclage au-dessus : extraction forcée
  F_ext_frc  : REAL := 45.0;   // Hz
  T_sort_sp  : REAL := 47.0;   // °C, sortie visée (boucle L5, active avec C1)
  F_sb       : REAL := 30.0;   // Hz, standby air chaud et brassage
  F_sb_min   : REAL := 25.0;   // Hz, mini recyclage / extraction en standby
  B_on       : REAL := 33.0;   // %, B1/B2 : marche au-dessus (recette « brûleur à point »)
  B_off      : REAL := 27.0;   // %, arrêt en dessous
  t_B_min    : REAL := 600.0;  // s, marche et arrêt mini de B1/B2
  V_rampe    : REAL := 0.1;    // Hz/s
END_VAR
VAR_OUTPUT
  Mode       : INT;            // 1 production, 2 attente mouleuse, 3 standby, 4 reprise, 5 repli
  Prod       : BOOL;           // pour FB104, FB106, FB107 (modes 1, 2, 4)
  ASC_SP     : REAL;           // °C -> consigne du thermorégulateur ASC 4-20 mA (SM332)
  ASC_ON     : BOOL;           // autorisation du brûleur ASC
  Reg_arr    : REAL;           // %, registre arrière ASC (M2)
  F_AC, F_REC, F_EXT, F_BR : REAL;   // Hz -> variateurs (Profibus)
  B1_ON, B2_ON : BOOL;
  T_HR       : REAL;           // °C, demande de la boucle humidité du recyclage
  T_dem      : REAL;           // °C, MAX(T_HR ; T_HRa)
  Gel        : BOOL;           // consigne limitée en aval
  Req_recup  : REAL;           // %, demande de récupération au S7-412
  Alm_ASC    : BOOL;           // ASC éteint en production plus de 5 min sans demande d'arrêt
END_VAR
VAR
```

```
  init       : BOOL;
  mode_old   : INT;
  t_moul     : REAL;           // s depuis l'arrêt de la mouleuse et de la chaîne
  t_rep      : REAL;
  I_HR       : REAL;
  I5         : REAL;           // correction extraction sur la sortie (L5)
  sp_r       : REAL;           // consigne ASC après rampe
  ac_t, rec_t, ext_t, br_t : REAL;   // cibles avant rampe
  ac_sb, rec_sb, ext_sb, br_sb : REAL;
  b1, b2     : BOOL;
  t_b1, t_b2 : REAL;
  t_noflam   : REAL;
END_VAR
VAR_TEMP
  e, P, T_cmd, a, f, d : REAL;
  bdem, b1d, b2d : BOOL;
END_VAR

IF NOT init THEN
  sp_r := ASC_act; F_AC := F_AC_act; F_REC := F_REC_act; F_EXT := F_EXT_act; F_BR := F_BR_act;
  ac_t := F_AC_act; rec_t := F_REC_act; ext_t := F_EXT_act; br_t := F_BR_act;
  Mode := 1; mode_old := 1; t_b1 := t_B_min; t_b2 := t_B_min; init := TRUE;
END_IF;

// ---------- 1. Modes de marche
IF Mouleuse OR Chaine THEN t_moul := 0.0; ELSE t_moul := t_moul + dt; END_IF;
IF Def_sonde THEN
  Mode := 5;
ELSIF Mouleuse OR Chaine THEN
  IF (mode_old = 3) OR ((mode_old = 4) AND (t_rep < t_rep_vent)) THEN
    IF mode_old = 3 THEN t_rep := 0.0; END_IF;
    Mode := 4;
  ELSE
    Mode := 1;
  END_IF;
ELSIF t_moul < t_standby THEN
  Mode := 2;                                  // attente : tout continue, valeurs figées
ELSE
  Mode := 3;                                  // standby régulé
END_IF;
IF Mode = 4 THEN t_rep := t_rep + dt; END_IF;
mode_old := Mode;
Prod := (Mode = 1) OR (Mode = 2) OR (Mode = 4) OR ((Mode = 5) AND (Mouleuse OR Chaine));

// ---------- 2. Boucle de l'humidité du recyclage (T_HR)
e := HR_rec - SP_HR;
IF ABS(e) <= DB_HR THEN e := 0.0; END_IF;
P := Kp_HR * e;
IF NOT Auto_ASC THEN
  I_HR := ASC_act - T0_HR - P;                // suivi : reprise sans à-coup
ELSIF (Mode = 1) AND HR_rec_ok AND NOT Gel THEN
  I_HR := I_HR + Kp_HR * dt / Ti_HR * e;
END_IF;
IF I_HR > 15.0 THEN I_HR := 15.0; ELSIF I_HR < -15.0 THEN I_HR := -15.0; END_IF;
T_HR := T0_HR + P + I_HR;
IF NOT HR_rec_ok THEN T_HR := T_min; END_IF;   // la boucle HRa (FB106) reste active
IF T_HR > T_max THEN T_HR := T_max; ELSIF T_HR < T_min THEN T_HR := T_min; END_IF;

// ---------- 3. Sélecteur de la consigne ASC
T_dem := T_HR;
IF T_HRa > T_dem THEN T_dem := T_HRa; END_IF;
T_cmd := T_dem;
IF T_L1 < T_cmd THEN T_cmd := T_L1; END_IF;
IF T_L4 < T_cmd THEN T_cmd := T_L4; END_IF;
IF T_L7 < T_cmd THEN T_cmd := T_L7; END_IF;
IF T_max < T_cmd THEN T_cmd := T_max; END_IF;
IF Mode = 5 THEN T_cmd := T_repli; IF T_L4 < T_cmd THEN T_cmd := T_L4; END_IF; END_IF;
Gel := T_cmd < T_dem - 0.5;
IF NOT Auto_ASC THEN
  sp_r := ASC_act;
ELSIF T_cmd > sp_r + V_up * dt / 60.0 THEN
  sp_r := sp_r + V_up * dt / 60.0;            // montée limitée
ELSE
  sp_r := T_cmd;                              // descente immédiate (protection)
END_IF;
ASC_SP := sp_r;

// ---------- 4. Autorisation de l'ASC et registre arrière
ASC_ON := ((Mode = 1) OR (Mode = 2) OR (Mode = 5) OR ((Mode = 4) AND (t_rep >= t_rep_asc)))
          AND NOT Stop_ASC AND NOT Stop_crit;
IF ASC_ON THEN
  a := M2_pos + M2_plus;
  IF a > 100.0 THEN a := 100.0; END_IF;
  Reg_arr := a;
ELSE
  Reg_arr := 0.0;                             // standby, zéro gaz : registre fermé (moins d'air froid)
END_IF;
IF (Mode = 1) AND ASC_ON AND NOT Flamme THEN t_noflam := t_noflam + dt; ELSE t_noflam := 0.0; END_IF;
Alm_ASC := t_noflam > 300.0;                  // cas du 16/10 22:15-22:58 (ASC éteint 43 min en production)

// ---------- 5. Ventilateurs : cibles
// standby régulé sur l'humidité présente (calculé en permanence pour la reprise)
ac_sb := F_sb; br_sb := F_sb;
f := F_sb + 1.25 * (HR_rec - 26.0);           // 22 % -> 25 Hz ; 30 % -> 35 Hz
IF f < F_sb_min THEN f := F_sb_min; ELSIF f > 40.0 THEN f := 40.0; END_IF;
rec_sb := f; ext_sb := f;
IF HRa_ok AND (HRa_f > 14.0) THEN rec_sb := rec_sb + 5.0; END_IF;

CASE Mode OF
  1, 4:
    // production : base FB104 (cadence, régime) + corrections
```

```
    f := F_AC_b + dF_AC;
    IF f < AC_min THEN f := AC_min; ELSIF f > AC_max THEN f := AC_max; END_IF;
    ac_t := f;
    rec_t := F_REC_b;
    IF HRa_ok AND (HRa_f > HRa_L4) THEN rec_t := rec_t + 5.0; END_IF;   // L4
    // L5 : sortie cheminée (seulement avec C1), lente, +-5 Hz
    IF Sortie_ok AND (Mode = 1) THEN
      d := -0.5 * (T_sortie - T_sort_sp);
      IF (HR_sortie > 80.0) AND (d < 0.0) THEN d := 0.0; END_IF;          // pas de baisse si l'air sort humide
      I5 := I5 + dt / 1200.0 * d;
      IF I5 > 5.0 THEN I5 := 5.0; ELSIF I5 < -5.0 THEN I5 := -5.0; END_IF;
    END_IF;
    ext_t := F_EXT_b + I5;
    IF HR_rec_ok AND (HR_rec > HR_ext_frc) THEN ext_t := F_ext_frc; END_IF;
    br_t := F_BR_b;
    IF Mode = 4 THEN                          // reprise : passage progressif standby -> production
      a := t_rep / t_rep_vent;
      IF a > 1.0 THEN a := 1.0; END_IF;
      ac_t  := ac_sb  + a * (ac_t  - ac_sb);
      rec_t := rec_sb + a * (rec_t - rec_sb);
      ext_t := ext_sb + a * (ext_t - ext_sb);
      br_t  := br_sb  + a * (br_t  - br_sb);
    END_IF;
  3:
    ac_t := ac_sb; rec_t := rec_sb; ext_t := ext_sb; br_t := br_sb;
END_CASE;                                      // modes 2 (attente) et 5 (repli) : cibles figées
IF ac_t  < F_min THEN ac_t  := F_min; ELSIF ac_t  > F_max THEN ac_t  := F_max; END_IF;
IF rec_t < F_min THEN rec_t := F_min; ELSIF rec_t > F_max THEN rec_t := F_max; END_IF;
IF ext_t < F_min THEN ext_t := F_min; ELSIF ext_t > F_max THEN ext_t := F_max; END_IF;
IF br_t  < F_min THEN br_t  := F_min; ELSIF br_t  > F_max THEN br_t  := F_max; END_IF;

// ---------- 6. Rampes 0,1 Hz/s (ou suivi en observation)
d := V_rampe * dt;
IF NOT Auto_VENT THEN
  F_AC := F_AC_act; F_REC := F_REC_act; F_EXT := F_EXT_act; F_BR := F_BR_act;
ELSE
  IF ac_t  > F_AC  + d THEN F_AC  := F_AC  + d; ELSIF ac_t  < F_AC  - d THEN F_AC  := F_AC  - d; ELSE F_AC  := ac_t;  END_IF;
  IF rec_t > F_REC + d THEN F_REC := F_REC + d; ELSIF rec_t < F_REC - d THEN F_REC := F_REC - d; ELSE F_REC := rec_t; END_IF;
  IF ext_t > F_EXT + d THEN F_EXT := F_EXT + d; ELSIF ext_t < F_EXT - d THEN F_EXT := F_EXT - d; ELSE F_EXT := ext_t; END_IF;
  IF br_t  > F_BR  + d THEN F_BR  := F_BR  + d; ELSIF br_t  < F_BR  - d THEN F_BR  := F_BR  - d; ELSE F_BR  := br_t;  END_IF;
END_IF;

// ---------- 7. B1 / B2 : logique actuelle 27/33 % conservée, temps mini, option O1
IF HR_rec_ok AND (HR_rec > B_on) THEN bdem := TRUE;
ELSIF (NOT HR_rec_ok) OR (HR_rec < B_off) THEN bdem := FALSE;
ELSE bdem := b2;                               // entre les seuils : état maintenu
END_IF;
b2d := (bdem OR (O1 AND B2_antic)) AND Prod AND NOT Stop_crit;
b1d := (bdem OR (O1 AND B1_antic AND b2)) AND Prod AND NOT Stop_crit;   // B1 jamais avant B2 en O1
t_b2 := t_b2 + dt; t_b1 := t_b1 + dt;
IF (b2d <> b2) AND (t_b2 >= t_B_min) THEN b2 := b2d; t_b2 := 0.0; END_IF;
IF (b1d <> b1) AND (t_b1 >= t_B_min) THEN b1 := b1d; t_b1 := 0.0; END_IF;
IF NOT Prod THEN b1 := FALSE; b2 := FALSE; END_IF;  // standby : brûleurs arrêtés (règle actuelle)
B1_ON := b1; B2_ON := b2;

// ---------- 8. Demande de récupération au four (L8)
a := Req_rec_b;
IF Gel AND Prod THEN a := 100.0; END_IF;       // consigne bridée : chaleur gratuite d'abord
IF a < 0.0 THEN a := 0.0; ELSIF a > 100.0 THEN a := 100.0; END_IF;
Req_recup := a;
END_FUNCTION_BLOCK
```

FB102 CADENCE (S7-300) — cadence et cycle réel

```
// ===================================================================
// FB102 CADENCE - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Cadence réelle du séchoir comptée sur les clefs des balancelles
// (64 balancelles x 5 clefs = 320 clefs par tour ; compteur « N/320 » du
// synoptique, ou impulsions de FB105). Remplace le débit de la mouleuse.
// Projet définitif V1 du 19/10/2026.
// Cycle réel de séchage = K_tour x durée d'un tour (K_tour fixé par E5 :
// un tour ≈ 50 min, séchage ≈ 2 h 30, soit K_tour ≈ 3).
// Appel : OB35 (100 ms), instance DB102, en premier.
// ===================================================================
FUNCTION_BLOCK FB102
VAR_INPUT
  dt       : REAL := 0.1;     // s
  N_clef   : INT;             // 0..319, compteur de clefs du synoptique
  K_tour   : REAL := 3.0;     // tours par cycle de séchage (essai E5)
  T_arret  : REAL := 180.0;   // s sans clef : chaîne considérée arrêtée
END_VAR
VAR_OUTPUT
  R10      : REAL;            // clefs/min, moyenne glissante 10 min (0 si chaîne arrêtée)
  R2       : REAL;            // clefs/min, moyenne 2 min (cadence instantanée)
  Tour_min : REAL;            // min, durée d'un tour de 320 clefs à la cadence R10
  Cycle_h  : REAL;            // h, cycle réel de séchage
  Cycle_ok : BOOL;            // cadence assez stable pour être utilisée
  Marche   : BOOL;            // chaîne en mouvement
  t_arr    : REAL;            // s depuis la dernière clef
END_VAR
VAR
  init    : BOOL;
  N_old   : INT;
  cnt     : REAL;
  t_m     : REAL;
  buf     : ARRAY[0..9] OF REAL;
  idx     : INT;
  nbuf    : INT;
END_VAR
VAR_TEMP
  dN : INT;
  k  : INT;
  s  : REAL;
END_VAR

IF NOT init THEN
  N_old := N_clef; FOR k := 0 TO 9 DO buf[k] := 0.0; END_FOR; init := TRUE;
END_IF;
dN := N_clef - N_old;
IF dN < 0 THEN dN := dN + 320; END_IF;        // passage 319 -> 0
IF dN > 20 THEN dN := 0; END_IF;               // remise à zéro du compteur : ignorée
N_old := N_clef;
cnt := cnt + INT_TO_REAL(dN);
IF dN > 0 THEN t_arr := 0.0; ELSE t_arr := t_arr + dt; END_IF;
Marche := t_arr < T_arret;

t_m := t_m + dt;
IF t_m >= 60.0 THEN
  buf[idx] := cnt; idx := (idx + 1) MOD 10; cnt := 0.0; t_m := 0.0;
  IF nbuf < 10 THEN nbuf := nbuf + 1; END_IF;
END_IF;
s := 0.0;
FOR k := 0 TO 9 DO s := s + buf[k]; END_FOR;
IF nbuf > 0 THEN R10 := s / INT_TO_REAL(nbuf); ELSE R10 := 0.0; END_IF;
s := buf[(idx + 9) MOD 10] + buf[(idx + 8) MOD 10];
R2 := s / 2.0;
IF NOT Marche THEN R10 := 0.0; R2 := 0.0; END_IF;

Cycle_ok := Marche AND (nbuf >= 5) AND (R10 >= 2.0);
IF R10 > 0.5 THEN
  Tour_min := 320.0 / R10;
  Cycle_h  := K_tour * Tour_min / 60.0;
ELSE
  Tour_min := 0.0; Cycle_h := 0.0;
END_IF;
END_FUNCTION_BLOCK
```

FB103 LIMITES (S7-300) — sondes, limites T°C1 / T°C4, registre arrière

```
// ===================================================================
// FB103 LIMITES - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Validité et filtrage des sondes de limite et de récupération ;
// limiteurs de la consigne ASC (T°C1 <= 145 °C, T°C4 <= 250 °C, plafond
// 135 °C) ; registre arrière de l'ASC (M2) commandé par T°C4 ;
// alarmes WinCC (pré-alarme, alarme, critique).
// Projet définitif V1 du 19/10/2026.
// Appel : OB35 (100 ms), instance DB103, AVANT FB104, FB106, FB107, FB101.
// ===================================================================
FUNCTION_BLOCK FB103
VAR_INPUT
  dt        : REAL := 0.1;     // s
  TC1       : REAL;            // °C, air soufflé (Pt100 0-160 °C)
  TC4       : REAL;            // °C, chambre de mélange (thermocouple K 0-360 °C)
  TC3       : REAL;            // °C, arrivée de la gaine de récupération (échelle à vérifier, étape 0)
  TC1_max   : REAL := 160.0;   // °C, haut d'échelle des sondes
  TC4_max   : REAL := 360.0;
  TC3_max   : REAL := 250.0;   // °C, échelle à confirmer (TC3 lu à 220-225 °C le 17/10)
  T_max     : REAL := 135.0;   // °C, plafond de la consigne ASC
  L1_seuil  : REAL := 140.0;   // °C, le limiteur T°C1 agit au-dessus
  L1_pente  : REAL := 3.0;     // °C de consigne par °C de T°C1
  L4_seuil  : REAL := 230.0;   // °C, le limiteur T°C4 agit au-dessus
  L4_pente  : REAL := 2.0;     // °C de consigne par °C de T°C4
  pos_min   : REAL := 30.0;    // %, ouverture mini du registre arrière quand l'ASC brûle (essai E6)
  ASC_brule : BOOL;            // flamme ASC présente
END_VAR
VAR_OUTPUT
  TC1f, TC4f, TC3f : REAL;     // °C, filtrés 30 s
  TC1_ok, TC4_ok, TC3_ok : BOOL;
  T_L1      : REAL;            // °C, limite de consigne venant de T°C1
  T_L4      : REAL;            // °C, limite de consigne venant de T°C4
  M2_pos    : REAL;            // %, position demandée du registre arrière (0 si ASC éteint)
  Stop_crit : BOOL;            // T°C4 >= 250 °C pendant 2 min : arrêt ASC demandé
  Pre_TC1, Alm_TC1, Crit_TC1 : BOOL;   // > 138, > 142, >= 145 °C
  Pre_TC4, Alm_TC4, Crit_TC4 : BOOL;   // > 230, > 240, >= 250 °C
  Def_sonde : BOOL;            // T°C1 ou T°C4 invalide : FB101 passe l'ASC à 110 °C
END_VAR
VAR
  init : BOOL;
  f10_1, f10_4, f10_3 : REAL;  // filtres 10 s (détection de saut)
  ref1, ref4, ref3    : REAL;  // valeur de référence pour la détection « figée »
  tfig1, tfig4, tfig3 : REAL;
  hold1 : BOOL; t_lib1 : REAL;
  t250  : REAL;
END_VAR
VAR_TEMP
  v : REAL;
END_VAR

IF NOT init THEN
  TC1f := TC1; TC4f := TC4; TC3f := TC3; f10_1 := TC1; f10_4 := TC4; f10_3 := TC3;
  ref1 := TC1; ref4 := TC4; ref3 := TC3; init := TRUE;
END_IF;

// ---------- 1. Filtres
TC1f := TC1f + dt / 30.0 * (TC1 - TC1f);
TC4f := TC4f + dt / 30.0 * (TC4 - TC4f);
TC3f := TC3f + dt / 30.0 * (TC3 - TC3f);
f10_1 := f10_1 + dt / 10.0 * (TC1 - f10_1);
f10_4 := f10_4 + dt / 10.0 * (TC4 - f10_4);
f10_3 := f10_3 + dt / 10.0 * (TC3 - f10_3);

// ---------- 2. Validité : hors échelle, saut brutal, valeur figée 10 min
IF ABS(TC1 - ref1) > 0.2 THEN ref1 := TC1; tfig1 := 0.0; ELSE tfig1 := tfig1 + dt; END_IF;
IF ABS(TC4 - ref4) > 0.5 THEN ref4 := TC4; tfig4 := 0.0; ELSE tfig4 := tfig4 + dt; END_IF;
IF ABS(TC3 - ref3) > 0.2 THEN ref3 := TC3; tfig3 := 0.0; ELSE tfig3 := tfig3 + dt; END_IF;
TC1_ok := (TC1 > 0.0) AND (TC1 < TC1_max - 0.5) AND (ABS(TC1 - f10_1) < 30.0) AND (tfig1 < 600.0);
TC4_ok := (TC4 > 0.0) AND (TC4 < TC4_max - 0.5) AND (ABS(TC4 - f10_4) < 80.0) AND (tfig4 < 600.0);
TC3_ok := (TC3 > 0.0) AND (TC3 < TC3_max - 0.5) AND (ABS(TC3 - f10_3) < 30.0) AND (tfig3 < 600.0);
Def_sonde := NOT (TC1_ok AND TC4_ok);

// ---------- 3. Limiteur T°C1 (L10)
v := T_max;
IF TC1f > L1_seuil THEN v := T_max - L1_pente * (TC1f - L1_seuil); END_IF;
IF TC1f >= 145.0 THEN hold1 := TRUE; t_lib1 := 0.0; END_IF;
IF hold1 THEN
  v := 100.0;                                  // action forte, libérée sous 142 °C pendant 5 min
  IF TC1f < 142.0 THEN t_lib1 := t_lib1 + dt; ELSE t_lib1 := 0.0; END_IF;
  IF t_lib1 >= 300.0 THEN hold1 := FALSE; END_IF;
END_IF;
IF NOT TC1_ok THEN v := 110.0; END_IF;
T_L1 := v;

// ---------- 4. Limiteur T°C4 (L11) et arrêt critique
v := T_max;
IF TC4f > L4_seuil THEN v := T_max - L4_pente * (TC4f - L4_seuil); END_IF;
IF TC4f >= 245.0 THEN v := 95.0; END_IF;       // consigne au minimum
IF NOT TC4_ok THEN v := 110.0; END_IF;
IF v < 95.0 THEN v := 95.0; END_IF;
T_L4 := v;
IF TC4_ok AND (TC4f >= 250.0) THEN t250 := t250 + dt; ELSE t250 := 0.0; END_IF;
Stop_crit := t250 >= 120.0;

// ---------- 5. Registre arrière ASC (M2) : jamais sous pos_min quand l'ASC brûle
IF ASC_brule THEN
  v := 60.0 + 4.0 * (TC4f - L4_seuil);
  IF v < pos_min THEN v := pos_min; END_IF;
  IF TC4f >= 245.0 THEN v := 100.0; END_IF;
  IF v > 100.0 THEN v := 100.0; END_IF;
```

```
  M2_pos := v;
ELSE
  M2_pos := 0.0;                              // fermé quand l'ASC est éteint (standby, zéro gaz)
END_IF;

// ---------- 6. Alarmes (dossier, tableau des alarmes)
Pre_TC1 := TC1f > 138.0; Alm_TC1 := TC1f > 142.0; Crit_TC1 := TC1f >= 145.0;
Pre_TC4 := TC4f > 230.0; Alm_TC4 := TC4f > 240.0; Crit_TC4 := TC4f >= 250.0;
END_FUNCTION_BLOCK
```

FB104 PLANIF_RECUP (S7-300) — récupération, régimes, fréquences de base

```
// ====================================================================
// FB104 PLANIF_RECUP - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Planification des ventilateurs et des brûleurs du séchoir
// en fonction de la température de récupération (TC3) et de la cadence.
// Projet définitif V1 du 19/10/2026 - valeurs de départ, à recaler par les essais E2, E10 et E13.
// Appel : OB35 (100 ms conseillé), instance DB104.
// Sorties = consignes de base. FB101 ajoute ensuite les corrections de
// sécurité (humidité avant recyclage, sortie cheminée, limites T°C1/T°C4),
// les rampes 0,1 Hz/s et les bornes 25-50 Hz.
// La gestion de flamme reste aux coffrets : ce bloc ne donne que des autorisations.
// ====================================================================
FUNCTION_BLOCK FB104
VAR_INPUT
  dt          : REAL := 0.1;    // s, période d'appel
  TC3         : REAL;           // °C, arrivée gaine récupération au séchoir
  TC3_ok      : BOOL;           // validité (FB103)
  T_dep       : REAL;           // °C, départ gaine (four, Profibus)
  T_dep_ok    : BOOL;
  dT_gaine    : REAL := 25.0;   // °C, chute départ -> arrivée (recalée par E2)
  Rec_pct     : REAL;           // %, vitesse récupération réelle (100 % = 50 Hz = 1500 tr/min)
  Rec_ok      : BOOL;
  Ta          : REAL := 30.0;   // °C, ambiante (sonde C2 à installer)
  Cycle_h     : REAL;           // h, cycle moyen réel (FB102, clefs balancelles)
  Cycle_ok    : BOOL;
  Gaz_Nm3h    : REAL;           // Nm3/h, débit corrigé compteur séchoir
  Gaz_ok      : BOOL;
  HR_rec      : REAL;           // %, humidité zone recyclage
  HRa         : REAL;           // %, humidité avant recyclage (limite 14 %)
  Production  : BOOL;           // balancelles en production (hors standby)
  Auto        : BOOL;           // FB104 actif ; FAUX = observation (sorties calculées, non utilisées)
END_VAR
VAR_OUTPUT
  TC3_eq      : REAL;           // °C, température équivalente à 750 tr/min
  Q_rec       : REAL;           // kW, apport de la gaine
  Q_need      : REAL;           // kW, besoin estimé à la cadence
  D           : REAL;           // charge brûleurs / capacité ASC (1,0 = ASC à fond)
  Regime      : INT;            // 1 = A récupération dominante, 2 = B mixte, 3 = C récupération faible
  F_AC        : REAL;           // Hz consigne de base air chaud
  F_REC       : REAL;           // Hz recyclage
  F_EXT       : REAL;           // Hz extraction air humide
  F_BR        : REAL;           // Hz brassage
  ASC_mini    : BOOL;           // régime A : ASC ramené au minimum puis arrêt autorisé
  B2_anticip  : BOOL;           // autorisation anticipée B2 (option O1, validée par l'exploitant)
  B1_anticip  : BOOL;
  Req_recup   : REAL;           // %, demande de récupération au four (L8, bornée par le four)
  M2_plus     : REAL;           // %, ouverture supplémentaire registre arrière ASC (anticipation)
  Alarme_cap  : BOOL;           // cadence trop haute pour la chaleur disponible
  Def_TC3     : BOOL;           // TC3 et départ gaine invalides : repli
END_VAR
VAR
  TC3f, TC3m10, Qrec_m10, Qneed_m10, Dmes_f, t_reg, t_cap, PI_I : REAL;
  q_p     : REAL := 1.05;       // kWh par pièce (adapté lentement)
  init    : BOOL;
END_VAR
VAR_TEMP
  T, rho, r, d, s, Nrel, Dpred, Dant, e, tmp, Trec_used : REAL;
  candidat : INT;
END_VAR
CONST
  V_REC   := 56300.0;   // m3/h à 750 tr/min
  ASC_MAX := 4070.0;    // kW utiles
  B_KW    := 631.0;     // kW utiles par brûleur arrière
  KWH_NM3 := 10.52;     // kWh utiles par Nm3
  PIECES  := 15360.0;
END_CONST

// ---------- 1. Température de récupération utilisée (TC3, sinon départ - chute de gaine)
IF TC3_ok THEN
  Trec_used := TC3; Def_TC3 := FALSE;
ELSIF T_dep_ok THEN
  Trec_used := T_dep - dT_gaine; Def_TC3 := FALSE;
ELSE
  Trec_used := TC3f; Def_TC3 := TRUE;      // dernière valeur filtrée, alarme
END_IF;
IF NOT init THEN
  TC3f := Trec_used; TC3m10 := Trec_used; init := TRUE;
END_IF;
TC3f   := TC3f   + dt / 30.0  * (Trec_used - TC3f);     // filtre 30 s
TC3m10 := TC3m10 + dt / 600.0 * (TC3f - TC3m10);        // moyenne glissante ~10 min

// ---------- 2. Puissance de la gaine et température équivalente
IF Rec_ok THEN Nrel := Rec_pct / 50.0; ELSE Nrel := 1.0; END_IF;   // 50 % = 750 tr/min
rho   := 353.0 / (TC3f + 273.15);
Q_rec := V_REC * Nrel * rho * 1.01 * (TC3f - Ta) / 3600.0;
IF Q_rec < 0.0 THEN Q_rec := 0.0; END_IF;
// TC3_eq : solution de Q(T, 750) = Q_rec  ->  T = (k*Ta + 273,15*q)/(k - q), k = V*353*1,01/3600
tmp := V_REC * 353.0 * 1.01 / 3600.0;
IF tmp - Q_rec > 1.0 THEN TC3_eq := (Q_rec * 273.15 + tmp * Ta) / (tmp - Q_rec); ELSE TC3_eq := 600.0; END_IF;

// ---------- 3. Besoin à la cadence
IF Cycle_ok AND Cycle_h > 1.5 THEN
  Q_need := q_p * PIECES / Cycle_h;  r := 2.5 / Cycle_h;
ELSE
  Q_need := q_p * PIECES / 2.67;     r := 0.94;    // repli : cycle 2 h 40
END_IF;
IF Qrec_m10 = 0.0 THEN Qrec_m10 := Q_rec; Qneed_m10 := Q_need; END_IF;
Qrec_m10  := Qrec_m10  + dt / 600.0 * (Q_rec  - Qrec_m10);
Qneed_m10 := Qneed_m10 + dt / 600.0 * (Q_need - Qneed_m10);
// ---------- 4. Charge brûleurs D = mesure (gaz) + anticipation (modèle)
```

```
Dpred := (Q_need - Q_rec) / ASC_MAX;
Dant  := ((Q_need - Qneed_m10) - (Q_rec - Qrec_m10)) / ASC_MAX;   // nulle en régime établi
IF Gaz_ok THEN
  Dmes_f := Dmes_f + dt / 120.0 * (Gaz_Nm3h * KWH_NM3 / ASC_MAX - Dmes_f);
  D := Dmes_f + Dant;
  // adaptation lente de q_p (constante 24 h), seulement en production établie
  IF Production AND Gaz_Nm3h > 50.0 AND ABS(Dant) < 0.05 THEN
    tmp := (Gaz_Nm3h * KWH_NM3 + Q_rec) * Cycle_h / PIECES;
    q_p := q_p + dt / 86400.0 * (tmp - q_p);
    IF q_p < 0.80 THEN q_p := 0.80; ELSIF q_p > 1.30 THEN q_p := 1.30; END_IF;
  END_IF;
ELSE
  D := Dpred;                    // compteur gaz indisponible : modèle seul
END_IF;

// ---------- 5. Régime (hystérésis + 15 min mini dans un régime)
candidat := Regime;
IF Regime = 0 THEN Regime := 2; candidat := 2; END_IF;
CASE Regime OF
  1: IF D > 0.35 THEN candidat := 2; END_IF;
  2: IF D > 0.95 THEN candidat := 3; ELSIF D < 0.20 AND HR_rec < 37.0 THEN candidat := 1; END_IF;
  3: IF D < 0.85 THEN candidat := 2; END_IF;
END_CASE;
IF NOT Production THEN candidat := 2; END_IF;          // standby et reprise gérés par FB101
IF candidat <> Regime THEN
  t_reg := t_reg + dt;
  IF t_reg >= 900.0 OR (candidat = 3 AND Dant > 0.10) THEN   // 15 min, ou tout de suite si la récupération chute vite
    Regime := candidat; t_reg := 0.0;
  END_IF;
ELSE
  t_reg := 0.0;
END_IF;

// ---------- 6. Fréquences de base (cadence) + corrections de régime
F_AC  := 45.0 + 20.0 * (r - 1.0);
F_REC := 45.0 + 20.0 * (r - 1.0);
F_EXT := 41.0 + 18.0 * (r - 1.0);
F_BR  := 47.0 + 15.0 * (r - 1.0);
ASC_mini := FALSE; B2_anticip := FALSE; B1_anticip := FALSE; M2_plus := 0.0;
CASE Regime OF
  1: // A : la récupération couvre presque tout. L'humidité est tenue par l'extraction (PI), l'ASC va au minimum.
     s := (0.25 - D) / 0.25; IF s < 0.0 THEN s := 0.0; ELSIF s > 1.0 THEN s := 1.0; END_IF;
     e := HR_rec - 36.5;
     IF ABS(e) <= 2.5 THEN e := 0.0; END_IF;
     PI_I := PI_I + dt / 600.0 * 0.5 * e;
     IF PI_I > 5.0 THEN PI_I := 5.0; ELSIF PI_I < -5.0 THEN PI_I := -5.0; END_IF;
     F_EXT := F_EXT - 2.0 * s + 0.5 * e + PI_I;       // HR haute -> extraction monte
     ASC_mini := TRUE;
     Req_recup := 100.0;                              // le four borne (limite F5)
  2: PI_I := 0.0;
     Req_recup := 100.0 * (D - 0.5);                  // demande croissante avec la charge ASC
  3: // C : récupération faible ou cadence haute. Garder la chaleur, préparer B2/B1, diluer la chambre.
     PI_I := 0.0;
     d := D - 0.9; IF d < 0.0 THEN d := 0.0; ELSIF d > 0.4 THEN d := 0.4; END_IF;
     F_EXT := F_EXT - 10.0 * d;                       // max -4 Hz
     F_REC := F_REC + 7.5 * d;                        // max +3 Hz
     F_AC  := F_AC  + 5.0 * d;                        // max +2 Hz
     M2_plus := 25.0 * d;                             // max +10 % registre arrière (T°C4)
     B2_anticip := D > 1.00;
     B1_anticip := D > 1.00 + B_KW / ASC_MAX;
     Req_recup := 100.0;
END_CASE;
IF Req_recup < 0.0 THEN Req_recup := 0.0; ELSIF Req_recup > 100.0 THEN Req_recup := 100.0; END_IF;
// Sécurité produit : l'extraction ne descend jamais si l'air avant recyclage est humide
IF HRa > 13.0 AND F_EXT < 41.0 + 18.0 * (r - 1.0) THEN F_EXT := 41.0 + 18.0 * (r - 1.0); END_IF;

// ---------- 7. Capacité dépassée (ASC + B1 + B2 insuffisants plus de 10 min)
IF D > 1.0 + 2.0 * B_KW / ASC_MAX THEN t_cap := t_cap + dt; ELSE t_cap := 0.0; END_IF;
Alarme_cap := t_cap > 600.0;

// ---------- 8. Bornes
IF F_AC  < 25.0 THEN F_AC  := 25.0; ELSIF F_AC  > 50.0 THEN F_AC  := 50.0; END_IF;
IF F_REC < 25.0 THEN F_REC := 25.0; ELSIF F_REC > 50.0 THEN F_REC := 50.0; END_IF;
IF F_EXT < 25.0 THEN F_EXT := 25.0; ELSIF F_EXT > 50.0 THEN F_EXT := 50.0; END_IF;
IF F_BR  < 35.0 THEN F_BR  := 35.0; ELSIF F_BR  > 50.0 THEN F_BR  := 50.0; END_IF;
IF NOT Auto THEN ASC_mini := FALSE; B2_anticip := FALSE; B1_anticip := FALSE; END_IF;
END_FUNCTION_BLOCK
```

FB105 BALANCELLE_BLOCAGE (S7-300) — balancelle au blocage

```
// ================================================================
// FB105 BALANCELLE_BLOCAGE - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Repère la balancelle (1..64) et la clef (1..5) en cours de montée
// quand l'ampérage de la chaîne dépasse la consigne MAX (blocage).
// Projet définitif V1 du 19/10/2026 - à valider par l'essai de repérage (balancelle témoin).
// Appel : OB35 (100 ms). DB d'instance RÉMANENT (survit aux coupures STEG).
// Capteurs : S_REF = détecteur inductif au poste de montée, cible fixée sur la balancelle n°1
//            S_CLEF = détecteur inductif au poste de montée, une impulsion par clef (secours : pics d'ampérage)
// ================================================================
FUNCTION_BLOCK FB105
VAR_INPUT
  dt         : REAL := 0.1;     // s
  I_chaine   : REAL;            // A, ampérage moteur chaîne (ACTUEL AMPER)
  I_max      : REAL := 21.0;    // A, consigne MAX AMPER (seuil de blocage)
  Chaine_run : BOOL;            // chaîne commandée en marche
  S_REF      : BOOL;            // cible balancelle n°1
  S_CLEF     : BOOL;            // passage d'une clef (si absent : FALSE et Use_pics := TRUE)
  Use_pics   : BOOL := FALSE;   // compter les clefs sur les pics d'ampérage (mode secours)
  I_pic      : REAL := 14.0;    // A, seuil de détection d'une montée en mode secours
  Ack        : BOOL;            // acquittement du défaut depuis WinCC
END_VAR
VAR_OUTPUT
  Balancelle : INT;             // 1..64, balancelle au poste de montée
  Clef       : INT;             // 1..5, clef de cette balancelle
  Bal_bloc   : INT;             // balancelle du dernier blocage
  Clef_bloc  : INT;
  Defaut     : BOOL;            // blocage mémorisé, à acquitter
  Ref_ok     : BOOL;            // comptage recalé sur la balancelle n°1 depuis le démarrage
  Err_comptage : BOOL;          // nombre de clefs entre deux références <> 320
END_VAR
VAR
  n_clef     : INT;             // clefs comptées depuis la dernière référence (0..319)
  clefs_tour : INT;             // clefs comptées au dernier tour complet
  f_ref, f_clef, f_pic, f_bloc : BOOL;
  t_sur      : REAL;            // durée au-dessus de I_max
  Imax_bloc  : REAL;
  // Journal des 50 derniers blocages (à archiver dans WinCC)
  Journal : ARRAY[1..50] OF STRUCT
      Date_heure : DATE_AND_TIME;
      Bal        : INT;
      Clf        : INT;
      I_crete    : REAL;
  END_STRUCT;
  idx        : INT := 0;
  Cpt_bal    : ARRAY[1..64] OF INT;   // nombre de blocages par balancelle (statistique)
END_VAR
VAR_TEMP
  impulsion : BOOL;
  dt_now    : DATE_AND_TIME;
  ret       : INT;
END_VAR

// ---- 1. Comptage des clefs (front montant)
IF Use_pics THEN
  impulsion := (I_chaine > I_pic) AND NOT f_pic;   // front montant d'un pic
  f_pic := I_chaine > I_pic - 1.0;                 // hystérésis 1 A
ELSE
  impulsion := S_CLEF AND NOT f_clef;
  f_clef := S_CLEF;
END_IF;
IF impulsion AND Chaine_run THEN
  n_clef := n_clef + 1;
  IF n_clef >= 320 THEN n_clef := 0; END_IF;        // 64 balancelles × 5 clefs
END_IF;

// ---- 2. Recalage sur la balancelle n°1
IF S_REF AND NOT f_ref THEN
  clefs_tour := n_clef;
  Err_comptage := Ref_ok AND (n_clef <> 0) AND (n_clef <> 319);   // tolérance 1 clef
  n_clef := 0; Ref_ok := TRUE;
END_IF;
f_ref := S_REF;

Balancelle := n_clef / 5 + 1;   // division entière
Clef       := n_clef MOD 5 + 1;

// ---- 3. Détection du blocage : ampérage > consigne pendant 0,5 s
IF I_chaine > I_max THEN t_sur := t_sur + dt; ELSE t_sur := 0.0; END_IF;
IF t_sur >= 0.5 AND NOT f_bloc THEN
  f_bloc := TRUE; Defaut := TRUE;
  Bal_bloc := Balancelle; Clef_bloc := Clef; Imax_bloc := I_chaine;
  ret := READ_CLK(CDT := dt_now);                  // SFC1
  idx := idx MOD 50 + 1;
  Journal[idx].Date_heure := dt_now;
  Journal[idx].Bal := Bal_bloc;
  Journal[idx].Clf := Clef_bloc;
  Journal[idx].I_crete := Imax_bloc;
  Cpt_bal[Bal_bloc] := Cpt_bal[Bal_bloc] + 1;
END_IF;
IF f_bloc AND I_chaine > Imax_bloc THEN            // garder la crête
  Imax_bloc := I_chaine; Journal[idx].I_crete := I_chaine;
END_IF;
IF I_chaine < I_max - 2.0 THEN f_bloc := FALSE; END_IF;
IF Ack THEN Defaut := FALSE; END_IF;
END_FUNCTION_BLOCK
```

FB106 REGUL_HRA (S7-300) — humidité avant recyclage

```
// ====================================================================
// FB106 REGUL_HRA - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Boucle de l'humidité AVANT RECYCLAGE (HRa) sur le brûleur ASC
// (modulant air/gaz) et correction rapide des ventilateurs d'air chaud.
// Projet définitif V1 du 19/10/2026. Valeurs de départ, à recaler par les essais E5, E11 et E12.
//
// Principe :
//   consigne ASC = anticipation (cadence + TC3) + PI(HRa - 12,5 %)
//   air chaud    = base FB104 + 5 Hz par point au-dessus de 12,5 %
//   (la température est le levier de fond, la ventilation le levier rapide)
//   Gel = VRAI (consigne bridée en aval par FB101) : intégrale gelée.
//
// Appel : OB35 (100 ms), instance DB106, APRES FB104 et AVANT FB107/FB101.
// Dans FB101 :
//   ASC  := MIN( MAX(T_HR ; DB106.T_HRa) ; T_L1 ; T_L4 ; DB107.T_L7 ; T_max )
//   f_AC := LIMIT(36, DB104.F_AC + DB106.dF_AC, 45)  puis rampe 0,1 Hz/s
// Les limiteurs T°C1 / T°C4 (FB103 : T_L1, T_L4) restent prioritaires.
// ====================================================================
FUNCTION_BLOCK FB106
VAR_INPUT
  dt       : REAL := 0.1;     // s, période d'appel
  HRa      : REAL;            // %, humidité zone avant recyclage
  HRa_ok   : BOOL;            // mesure valide (hors 0 de perte de liaison)
  N_clef   : INT;             // compteur de clefs 0..319 (synoptique « N/320 »)
  Prod     : BOOL;            // séchoir en production (hors standby)
  TC3f     : REAL;            // °C, TC3 filtrée (FB103 / FB104)
  TC3_ok   : BOOL;
  ASC_act  : REAL;            // °C, consigne ASC réellement envoyée (suivi sans à-coup)
  Auto     : BOOL;            // FAUX = observation : calcul seul, suivi de ASC_act
  Gel      : BOOL;            // VRAI = consigne limitée en aval (FB101) : intégrale gelée
  SP_HRa   : REAL := 12.5;    // %, consigne d'humidité avant recyclage
  Kp       : REAL := 2.5;     // °C de consigne ASC par point d'humidité
  Ti       : REAL := 480.0;   // s, temps d'intégrale (8 min)
  T_min    : REAL := 110.0;   // °C, consigne ASC mini
  T_max    : REAL := 135.0;   // °C, consigne ASC maxi (sous les limiteurs T°C1/T°C4)
  V_up     : REAL := 3.0;     // °C/min, montée maxi de la consigne
  V_dn     : REAL := 1.0;     // °C/min, descente maxi
  T_base0  : REAL := 112.0;   // °C, base chaîne arrêtée
  K_cad    : REAL := 1.5;     // °C par clef/min de cadence
  K_tc3    : REAL := 0.3;     // °C par °C de TC3 sous TC3_ref
  TC3_ref  : REAL := 180.0;   // °C
  K_ac     : REAL := 5.0;     // Hz d'air chaud par point d'humidité
  SP_ac    : REAL := 12.5;    // %, point neutre de la correction air chaud
  T_arret  : REAL := 180.0;   // s sans clef = chaîne arrêtée (ralenti)
  HRa_alm  : REAL := 13.5;    // %, seuil d'alarme opérateur
END_VAR
VAR_OUTPUT
  T_HRa    : REAL;            // °C, consigne ASC demandée par la boucle HRa
  dF_AC    : REAL;            // Hz, correction air chaud (-4 .. +6)
  R10      : REAL;            // clefs/min, cadence moyenne sur 10 min
  Ralenti  : BOOL;            // chaîne arrêtée depuis plus de T_arret
  HRa_f    : REAL;            // %, humidité filtrée 60 s
  T_ff     : REAL;            // °C, part anticipation
  Alarme   : BOOL;            // HRa > HRa_alm pendant 2 min
END_VAR
VAR
  init     : BOOL;
  N_old    : INT;
  cnt_min  : REAL;            // clefs comptées dans la minute en cours
  t_minute : REAL;
  buf      : ARRAY[0..9] OF REAL;   // clefs par minute, 10 dernières minutes
  idx      : INT;
  nbuf     : INT;
  t_noclef : REAL;
  I_term   : REAL;
  T_out    : REAL;
  t_alarm  : REAL;
END_VAR
VAR_TEMP
  dN      : INT;
  k       : INT;
  s       : REAL;
  e       : REAL;
  P_term  : REAL;
  T_cmd   : REAL;
  step_up : REAL;
  step_dn : REAL;
END_VAR

// ---------- 0. Initialisation
IF NOT init THEN
  N_old := N_clef; HRa_f := HRa; T_out := ASC_act; I_term := 0.0;
  FOR k := 0 TO 9 DO buf[k] := 0.0; END_FOR;
  nbuf := 0; idx := 0; init := TRUE;
END_IF;

// ---------- 1. Humidité avant recyclage filtrée 60 s (gelée si mesure invalide)
IF HRa_ok THEN
  HRa_f := HRa_f + dt / 60.0 * (HRa - HRa_f);
END_IF;

// ---------- 2. Cadence : clefs par minute, moyenne glissante 10 min
dN := N_clef - N_old;
IF dN < 0 THEN dN := dN + 320; END_IF;     // passage 319 -> 0
IF dN > 20 THEN dN := 0; END_IF;            // saut anormal (remise à zéro) ignoré
N_old := N_clef;
cnt_min := cnt_min + INT_TO_REAL(dN);
IF dN > 0 THEN t_noclef := 0.0; ELSE t_noclef := t_noclef + dt; END_IF;
t_minute := t_minute + dt;
```

```
IF t_minute >= 60.0 THEN
  buf[idx] := cnt_min; idx := (idx + 1) MOD 10;
  cnt_min := 0.0; t_minute := 0.0;
  IF nbuf < 10 THEN nbuf := nbuf + 1; END_IF;
END_IF;
s := 0.0;
FOR k := 0 TO 9 DO s := s + buf[k]; END_FOR;
IF nbuf > 0 THEN R10 := s / INT_TO_REAL(nbuf); ELSE R10 := 6.5; END_IF;
Ralenti := Prod AND (t_noclef > T_arret);
IF Ralenti THEN R10 := 0.0; END_IF;          // chaîne arrêtée : plus de charge d'eau

// ---------- 3. Anticipation : cadence + baisse de la chaleur de récupération
T_ff := T_base0 + K_cad * R10;
IF TC3_ok AND (TC3f < TC3_ref) THEN
  T_ff := T_ff + K_tc3 * (TC3_ref - TC3f);
END_IF;

// ---------- 4. PI sur l'humidité avant recyclage (anti-emballement par blocage)
e := HRa_f - SP_HRa;                          // e > 0 : trop humide -> plus chaud
P_term := Kp * e;
IF NOT Auto THEN
  // observation : l'intégrale suit la consigne réelle (reprise sans à-coup)
  I_term := ASC_act - T_ff - P_term;
  T_out  := ASC_act;
ELSE
  T_cmd := T_ff + P_term + I_term;
  IF NOT (Gel OR ((T_cmd >= T_max) AND (e > 0.0)) OR ((T_cmd <= T_min) AND (e < 0.0))) THEN
    I_term := I_term + Kp * dt / Ti * e;
  END_IF;
END_IF;
IF I_term > 15.0 THEN I_term := 15.0; ELSIF I_term < -15.0 THEN I_term := -15.0; END_IF;
T_cmd := T_ff + P_term + I_term;
IF T_cmd > T_max THEN T_cmd := T_max; ELSIF T_cmd < T_min THEN T_cmd := T_min; END_IF;

// ---------- 5. Vitesse de variation (montée plus rapide que la descente)
step_up := V_up * dt / 60.0;
step_dn := V_dn * dt / 60.0;
IF Auto THEN
  IF T_cmd > T_out + step_up THEN
    T_out := T_out + step_up;
  ELSIF T_cmd < T_out - step_dn THEN
    T_out := T_out - step_dn;
  ELSE
    T_out := T_cmd;
  END_IF;
END_IF;
T_HRa := T_out;

// ---------- 6. Correction rapide de l'air chaud (s'efface quand l'ASC a repris)
dF_AC := K_ac * (HRa_f - SP_ac);
IF dF_AC > 6.0 THEN dF_AC := 6.0; ELSIF dF_AC < -4.0 THEN dF_AC := -4.0; END_IF;
IF NOT HRa_ok THEN dF_AC := 0.0; END_IF;

// ---------- 7. Alarme opérateur (avant la limite de 14 %)
IF HRa_f > HRa_alm THEN t_alarm := t_alarm + dt; ELSE t_alarm := 0.0; END_IF;
Alarme := t_alarm > 120.0;
END_FUNCTION_BLOCK
```

FB107 ASC_PILOTE (S7-300) — reprise, plafond T°C4, zéro gaz

```
// ================================================================
// FB107 ASC_PILOTE - Séchoir B12 - S7-300 (CPU 317) - STEP 7 SCL
// Pilotage du brûleur ASC au-dessus de la boucle d'humidité :
//   1. reprise douce après un arrêt (consigne bornée à 115 °C, puis rampe)
//   2. plafond lent de la consigne sur l'ENVELOPPE de T°C4 (maximum sur
//      12 min) : coupe les pics du pompage sans pomper avec lui
//   3. détection du pompage de T°C4 (amplitude, période) -> alarme
//   4. mode « zéro gaz » : ASC arrêté et registre arrière fermé quand la
//      récupération suffit, avec temps mini d'arrêt et de marche
//   5. comptage des démarrages de flamme (cycles courts)
// Projet définitif V1 du 19/10/2026. Valeurs de départ, à recaler par les essais E8, E14 et E15.
//
// Appel : OB35 (100 ms), instance DB107, APRES FB103, FB104, FB106 et
// AVANT FB101. Dans FB101 :
//   T_cons := MIN( MAX(T_HR ; DB106.T_HRa) ; T_L1 ; T_L4 ; DB107.T_L7 ; 135 )
//   ASC_ON := ... AND NOT DB107.Stop_ASC  (Stop_ASC -> mode recette
//             « OFF BRUL + FERM REG » : brûleur arrêté, registre fermé)
// La gestion de flamme reste au calculateur du brûleur : ce bloc ne donne
// qu'une limite de consigne et une demande d'arrêt.
// ================================================================
FUNCTION_BLOCK FB107
VAR_INPUT
  dt         : REAL := 0.1;    // s, période d'appel
  Auto       : BOOL;           // FAUX = observation : calcul, alarmes, mais T_L7 = 135 et pas d'arrêt
  Prod       : BOOL;           // séchoir en production (FB101 : mode production ou attente)
  TC4        : REAL;           // °C, chambre de mélange filtrée (FB103)
  TC4_ok     : BOOL;
  TC1        : REAL;           // °C, air soufflé filtré (FB103)
  TC1_ok     : BOOL;
  HRa_f      : REAL;           // %, humidité avant recyclage filtrée (DB106)
  HRa_ok     : BOOL;
  T_dem      : REAL;           // °C, consigne demandée = MAX(T_HR ; T_HRa) (FB101, cycle précédent)
  ASC_mini   : BOOL;           // FB104 : régime A (récupération dominante)
  Regime     : INT;            // FB104 : 1 A, 2 B, 3 C
  Flamme     : BOOL;           // retour du calculateur Siemens : flamme ASC présente
  // ---- reprise
  T_reprise  : REAL := 115.0;  // °C, consigne maxi pendant la reprise
  t_reprise  : REAL := 1200.0; // s, durée du palier de reprise (20 min)
  V_reprise  : REAL := 1.0;    // °C/min, rampe après le palier
  T_libre    : REAL := 135.0;  // °C, au-delà la limite n'agit plus
  // ---- plafond sur l'enveloppe de T°C4
  TC4_haut   : REAL := 240.0;  // °C, enveloppe au-dessus : plafond -2 °C par minute
  TC4_bas    : REAL := 225.0;  // °C, enveloppe en dessous : plafond +1 °C par minute
  Cap_min    : REAL := 100.0;  // °C, plancher du plafond
  dCap_max   : REAL := 10.0;   // °C, retrait maxi du plafond sous la demande d'humidité
  // ---- pompage
  H_pomp     : REAL := 10.0;   // °C, hystérésis de détection des extrêmes
  A_pomp     : REAL := 60.0;   // °C, amplitude crête-creux comptée comme pompage
  N_pomp     : INT  := 3;      // cycles consécutifs pour l'alarme
  // ---- zéro gaz
  HRa_off    : REAL := 12.0;   // %, en dessous (avec demande au mini) : arrêt possible
  T_dem_off  : REAL := 112.0;  // °C, demande au plancher (T_min FB106 = 110)
  t_off      : REAL := 900.0;  // s, conditions d'arrêt tenues 15 min
  HRa_on     : REAL := 13.0;   // %, au-dessus pendant t_on : redémarrage
  t_on       : REAL := 120.0;  // s
  HRa_urg    : REAL := 13.5;   // %, redémarrage immédiat (ignore le temps mini)
  TC1_min    : REAL := 105.0;  // °C, air soufflé trop froid : redémarrage immédiat
  t_min_arr  : REAL := 1200.0; // s, arrêt mini (20 min)
  t_min_mar  : REAL := 1200.0; // s, marche mini avant un nouvel arrêt (20 min)
  Dem_max_h  : INT  := 4;      // démarrages de flamme par heure avant alarme
END_VAR
VAR_OUTPUT
  T_L7       : REAL;           // °C, limite de consigne ASC (entre dans le MIN de FB101)
  Stop_ASC   : BOOL;           // demande d'arrêt « zéro gaz »
  Etat       : INT;            // 0 libre, 1 reprise, 2 plafond T°C4, 3 zéro gaz
  TC4_env    : REAL;           // °C, maximum de T°C4 sur 12 min
  Cap        : REAL;           // °C, plafond actuel
  Pompage    : BOOL;           // alarme : T°C4 pompe (thermorégulateur ou charge mini)
  Ampl       : REAL;           // °C, dernière amplitude crête-creux de T°C4
  Periode    : REAL;           // min, dernière période crête-crête
  Dem_h      : INT;            // démarrages de flamme sur la dernière heure
  Alm_cycles : BOOL;           // trop de démarrages
END_VAR
VAR
  init       : BOOL;
  prod_old   : BOOL;
  rep_act    : BOOL;
  rep_lim    : REAL;
  t_rep      : REAL;
  buck       : ARRAY[0..11] OF REAL;   // maximum de T°C4 par minute, 12 dernières minutes
  ib         : INT;
  t_min      : REAL;
  dir        : INT;                    // 1 montée, -1 descente
  ext        : REAL;
  last_max   : REAL;
  last_min   : REAL;
  t_max_cnt  : REAL;
  n_cyc      : INT;
  zg         : BOOL;
  t_c_off    : REAL;
  t_c_on     : REAL;
  t_etat     : REAL;
  fl_old     : BOOL;
  dem        : ARRAY[0..5] OF INT;     // démarrages par tranche de 10 min
  id         : INT;
  t_10       : REAL;
END_VAR
VAR_TEMP
  k      : INT;
```

```
  m      : REAL;
  urg    : BOOL;
  c_off  : BOOL;
  lim    : REAL;
END_VAR

// ---------- 0. Initialisation
IF NOT init THEN
  FOR k := 0 TO 11 DO buck[k] := TC4; END_FOR;
  FOR k := 0 TO 5 DO dem[k] := 0; END_FOR;
  Cap := T_libre; rep_lim := T_libre; ext := TC4; last_max := TC4; last_min := TC4;
  dir := 1; t_etat := t_min_mar; prod_old := Prod; init := TRUE;
END_IF;

// ---------- 1. Reprise douce (front montant de la production)
IF Prod AND NOT prod_old THEN
  rep_act := TRUE; rep_lim := T_reprise; t_rep := 0.0;
END_IF;
prod_old := Prod;
IF rep_act THEN
  t_rep := t_rep + dt;
  IF t_rep > t_reprise THEN
    rep_lim := rep_lim + V_reprise * dt / 60.0;
  END_IF;
  IF (rep_lim >= T_libre) OR NOT Prod THEN
    rep_act := FALSE; rep_lim := T_libre;
  END_IF;
END_IF;

// ---------- 2. Enveloppe de T°C4 (maximum glissant sur 12 min)
IF TC4_ok THEN
  IF TC4 > buck[ib] THEN buck[ib] := TC4; END_IF;
END_IF;
m := buck[0];
FOR k := 1 TO 11 DO
  IF buck[k] > m THEN m := buck[k]; END_IF;
END_FOR;
TC4_env := m;

// ---------- 3. Plafond intégrateur, ajusté une fois par minute
t_min := t_min + dt;
IF t_min >= 60.0 THEN
  t_min := 0.0;
  IF TC4_ok AND Prod THEN
    IF TC4_env > TC4_haut THEN
      Cap := Cap - 2.0;
    ELSIF TC4_env < TC4_bas THEN
      Cap := Cap + 1.0;
    END_IF;
  ELSE
    Cap := Cap + 1.0;                    // pas de mesure ou arrêt : le plafond se relâche
  END_IF;
  // le plafond ne retire jamais plus de dCap_max à la demande d'humidité :
  // au-delà, la protection de T°C4 est assurée par FB103 (T_L4, registre M2)
  IF Cap < T_dem - dCap_max THEN Cap := T_dem - dCap_max; END_IF;
  IF Cap > T_libre THEN Cap := T_libre; ELSIF Cap < Cap_min THEN Cap := Cap_min; END_IF;
  ib := (ib + 1) MOD 12;                 // nouvelle case pour la minute suivante
  IF TC4_ok THEN buck[ib] := TC4; ELSE buck[ib] := 0.0; END_IF;
END_IF;

// ---------- 4. Détection du pompage (extrêmes avec hystérésis)
t_max_cnt := t_max_cnt + dt;
IF TC4_ok AND Prod THEN
  IF dir = 1 THEN
    IF TC4 > ext THEN
      ext := TC4;
    ELSIF TC4 < ext - H_pomp THEN         // un maximum vient d'être passé
      last_max := ext; Periode := t_max_cnt / 60.0; t_max_cnt := 0.0;
      Ampl := last_max - last_min;
      IF (Ampl >= A_pomp) AND (Periode >= 3.0) AND (Periode <= 25.0) THEN
        n_cyc := n_cyc + 1;
      ELSE
        n_cyc := 0;
      END_IF;
      dir := -1; ext := TC4;
    END_IF;
  ELSE
    IF TC4 < ext THEN
      ext := TC4;
    ELSIF TC4 > ext + H_pomp THEN         // un minimum vient d'être passé
      last_min := ext; dir := 1; ext := TC4;
    END_IF;
  END_IF;
END_IF;
IF t_max_cnt > 1800.0 THEN n_cyc := 0; END_IF;   // plus d'oscillation depuis 30 min
IF n_cyc > 100 THEN n_cyc := 100; END_IF;
Pompage := n_cyc >= N_pomp;

// ---------- 5. Mode zéro gaz (arrêt de l'ASC quand la récupération suffit)
t_etat := t_etat + dt;
c_off := Prod AND HRa_ok AND TC1_ok
         AND (HRa_f < HRa_off)
         AND ((T_dem <= T_dem_off) OR ASC_mini)
         AND (TC1 > TC1_min + 10.0)
         AND (Regime <> 3);
urg := (HRa_ok AND (HRa_f > HRa_urg)) OR (TC1_ok AND (TC1 < TC1_min)) OR (Regime = 3) OR NOT HRa_ok;
IF NOT Prod THEN
  zg := FALSE; t_c_off := 0.0; t_c_on := 0.0;   // standby : géré par FB101
ELSIF NOT zg THEN
  IF c_off THEN t_c_off := t_c_off + dt; ELSE t_c_off := 0.0; END_IF;
  IF (t_c_off >= t_off) AND (t_etat >= t_min_mar) THEN
```

```
    zg := TRUE; t_etat := 0.0; t_c_off := 0.0;
  END_IF;
ELSE
  IF HRa_ok AND (HRa_f > HRa_on) THEN t_c_on := t_c_on + dt; ELSE t_c_on := 0.0; END_IF;
  IF urg OR ((t_c_on >= t_on) AND (t_etat >= t_min_arr)) THEN
    zg := FALSE; t_etat := 0.0; t_c_on := 0.0;
    rep_act := TRUE; rep_lim := T_reprise; t_rep := t_reprise;   // rallumage : rampe sans palier
  END_IF;
END_IF;

// ---------- 6. Démarrages de flamme sur 1 h (6 tranches de 10 min)
IF Flamme AND NOT fl_old THEN dem[id] := dem[id] + 1; END_IF;
fl_old := Flamme;
t_10 := t_10 + dt;
IF t_10 >= 600.0 THEN
  t_10 := 0.0; id := (id + 1) MOD 6; dem[id] := 0;
END_IF;
Dem_h := 0;
FOR k := 0 TO 5 DO Dem_h := Dem_h + dem[k]; END_FOR;
Alm_cycles := Dem_h > Dem_max_h;

// ---------- 7. Sorties
lim := Cap;
IF rep_lim < lim THEN lim := rep_lim; END_IF;
IF zg THEN
  Etat := 3;
ELSIF rep_act AND (rep_lim <= Cap) THEN
  Etat := 1;
ELSIF Cap < T_libre THEN
  Etat := 2;
ELSE
  Etat := 0;
END_IF;
IF Auto THEN
  T_L7 := lim; Stop_ASC := zg;
ELSE
  T_L7 := T_libre; Stop_ASC := FALSE;      // observation : rien n'est imposé
END_IF;
END_FUNCTION_BLOCK
```

FB114 PRESSION_PORTES (S7-412) — pression et portes du four

```
// ================================================================
// FB114 PRESSION_PORTES - Four B12 - S7-412 - STEP 7 SCL
// Pression du four (consigne 0,6 mbar) :
//   - contre-pression = anticipation FB112 + PI rapide sur la pression
//   - tirage = base + recentrage lent (le tirage ramène la contre-pression
//     au milieu de sa plage : pas deux intégrales sur la même mesure)
//   - portes : anti-rebond 2 s des fins de course, pas d'anticipation
//     dès l'ouverture, intégrale gelée, retour par rampe après fermeture
// Projet définitif V1 du 19/10/2026. Signaux des portes déjà présents dans le S7-412.
// Appel : OB35 (100 ms), instance DB114, APRES FB112.
// Pas d'anticipation (Pas_*) et gains : valeurs de départ, essai E17.
// ================================================================
FUNCTION_BLOCK FB114
VAR_INPUT
  dt        : REAL := 0.1;     // s
  Auto      : BOOL;            // FAUX = observation (sorties = valeurs réelles)
  P_four    : REAL;            // mbar, pression de zone
  P_ok      : BOOL;
  SP        : REAL := 0.6;     // mbar
  Porte_S   : BOOL;            // porte de sortie ouverte (signal brut)
  Porte_E   : BOOL;            // porte d'entrée ouverte (signal brut)
  CP_ff     : REAL;            // %, contre-pression d'anticipation (FB112, tableau des paliers)
  TIR_base  : REAL;            // %, tirage de base (recette)
  CP_act    : REAL;            // %, contre-pression réelle (suivi)
  TIR_act   : REAL;            // %, tirage réel (suivi)
  Kp_cp     : REAL := 4.0;     // % de contre-pression par mbar
  Ti_cp     : REAL := 60.0;    // s
  T_vpc     : REAL := 600.0;   // s, constante du recentrage par le tirage
  CP_min    : REAL := 30.0;    // % (bande élargie ; limite haute selon l'essai E16)
  CP_max    : REAL := 90.0;
  TIR_min   : REAL := 55.0;    // %
  TIR_max   : REAL := 85.0;
  Pas_S_cp  : REAL := 5.0;     // %, porte de sortie : contre-pression +
  Pas_S_tir : REAL := -3.0;    // %, porte de sortie : tirage -
  Pas_E_cp  : REAL := -5.0;    // %, porte d'entrée : contre-pression -
  Pas_E_tir : REAL := 3.0;     // %, porte d'entrée : tirage +
  T_filtre  : REAL := 2.0;     // s, anti-rebond des fins de course (parasites de l'archive FC port)
  t_post    : REAL := 30.0;    // s, maintien après fermeture
  t_retour  : REAL := 45.0;    // s, rampe de retour
  V_max     : REAL := 2.0;     // %/s, variation maxi des sorties
END_VAR
VAR_OUTPUT
  CP_pct    : REAL;            // %, consigne variateur contre-pression (100 % = 50 Hz)
  TIR_pct   : REAL;            // %, consigne variateur tirage
  P_f       : REAL;            // mbar, pression filtrée 5 s
  Etat      : INT;             // 0 régulation, 1 porte sortie, 2 porte entrée, 3 retour, 4 défaut mesure
  PS_f, PE_f : BOOL;           // portes filtrées
END_VAR
VAR
  init : BOOL;
  I_cp, trim_tir : REAL;
  tS_on, tS_off, tE_on, tE_off : REAL;
  t_apres : REAL;
  k_cp, k_tir : REAL;          // pas d'anticipation en cours
  cp_o, tir_o : REAL;
END_VAR
VAR_TEMP
  e, u_cp, u_tir, dcp, s : REAL;
  porte : BOOL;
END_VAR

IF NOT init THEN
  P_f := P_four; cp_o := CP_act; tir_o := TIR_act; I_cp := CP_act - CP_ff; t_apres := 999.0; init := TRUE;
END_IF;

// ---------- 1. Mesure filtrée et anti-rebond des portes
IF P_ok THEN P_f := P_f + dt / 5.0 * (P_four - P_f); END_IF;
IF Porte_S THEN tS_on := tS_on + dt; tS_off := 0.0; ELSE tS_off := tS_off + dt; tS_on := 0.0; END_IF;
IF Porte_E THEN tE_on := tE_on + dt; tE_off := 0.0; ELSE tE_off := tE_off + dt; tE_on := 0.0; END_IF;
IF tS_on >= T_filtre THEN PS_f := TRUE; ELSIF tS_off >= T_filtre THEN PS_f := FALSE; END_IF;
IF tE_on >= T_filtre THEN PE_f := TRUE; ELSIF tE_off >= T_filtre THEN PE_f := FALSE; END_IF;
porte := PS_f OR PE_f;

// ---------- 2. Pas d'anticipation : appliqués à l'ouverture, retirés par rampe
IF porte THEN
  t_apres := 0.0;
  k_cp := 0.0; k_tir := 0.0;
  IF PS_f THEN k_cp := k_cp + Pas_S_cp; k_tir := k_tir + Pas_S_tir; END_IF;
  IF PE_f THEN k_cp := k_cp + Pas_E_cp; k_tir := k_tir + Pas_E_tir; END_IF;
ELSE
  t_apres := t_apres + dt;
  IF t_apres > t_post THEN
    s := dt / t_retour;
    IF k_cp > 0.0 THEN k_cp := k_cp - ABS(Pas_S_cp) * s; IF k_cp < 0.0 THEN k_cp := 0.0; END_IF;
    ELSIF k_cp < 0.0 THEN k_cp := k_cp + ABS(Pas_E_cp) * s; IF k_cp > 0.0 THEN k_cp := 0.0; END_IF; END_IF;
    IF k_tir > 0.0 THEN k_tir := k_tir - ABS(Pas_E_tir) * s; IF k_tir < 0.0 THEN k_tir := 0.0; END_IF;
    ELSIF k_tir < 0.0 THEN k_tir := k_tir + ABS(Pas_S_tir) * s; IF k_tir > 0.0 THEN k_tir := 0.0; END_IF; END_IF;
  END_IF;
END_IF;

// ---------- 3. PI de pression sur la contre-pression (pression haute -> moins d'air soufflé)
e := SP - P_f;                                  // e > 0 : pression trop basse -> plus de contre-pression
IF NOT P_ok THEN
  Etat := 4; e := 0.0;
ELSIF PS_f THEN Etat := 1;
ELSIF PE_f THEN Etat := 2;
ELSIF t_apres <= t_post + t_retour THEN Etat := 3;
ELSE Etat := 0;
END_IF;
```

```
u_cp := CP_ff + Kp_cp * e + I_cp + k_cp;
IF Auto AND (Etat = 0) THEN                    // intégrale gelée portes ouvertes, au retour et en défaut
  IF NOT (((u_cp >= CP_max) AND (e > 0.0)) OR ((u_cp <= CP_min) AND (e < 0.0))) THEN
    I_cp := I_cp + Kp_cp * dt / Ti_cp * e;
  END_IF;
END_IF;
IF NOT Auto THEN I_cp := CP_act - CP_ff - Kp_cp * e; END_IF;
IF I_cp > 25.0 THEN I_cp := 25.0; ELSIF I_cp < -25.0 THEN I_cp := -25.0; END_IF;
u_cp := CP_ff + Kp_cp * e + I_cp + k_cp;
IF u_cp > CP_max THEN u_cp := CP_max; ELSIF u_cp < CP_min THEN u_cp := CP_min; END_IF;

// ---------- 4. Tirage : recentrage lent de la contre-pression
dcp := u_cp - (CP_min + CP_max) / 2.0;          // écart au milieu de la plage
IF Auto AND (Etat = 0) THEN
  trim_tir := trim_tir - dt / T_vpc * 0.5 * dcp; // contre-pression basse durablement -> plus de tirage
END_IF;
IF trim_tir > 15.0 THEN trim_tir := 15.0; ELSIF trim_tir < -15.0 THEN trim_tir := -15.0; END_IF;
IF NOT Auto THEN trim_tir := TIR_act - TIR_base; END_IF;
u_tir := TIR_base + trim_tir + k_tir;
IF u_tir > TIR_max THEN u_tir := TIR_max; ELSIF u_tir < TIR_min THEN u_tir := TIR_min; END_IF;

// ---------- 5. Vitesse de variation, sorties
IF Auto THEN
  s := V_max * dt;
  IF u_cp > cp_o + s THEN cp_o := cp_o + s; ELSIF u_cp < cp_o - s THEN cp_o := cp_o - s; ELSE cp_o := u_cp; END_IF;
  IF u_tir > tir_o + s THEN tir_o := tir_o + s; ELSIF u_tir < tir_o - s THEN tir_o := tir_o - s; ELSE tir_o := u_tir; END_IF;
ELSE
  cp_o := CP_act; tir_o := TIR_act;
END_IF;
CP_pct := cp_o; TIR_pct := tir_o;
END_FUNCTION_BLOCK
```

FB115 CENTRALE_MOD (S7-412) — modulation air/gaz des centrales

```
// ====================================================================
// FB115 CENTRALE_MOD - Four B12 - S7-412 - STEP 7 SCL
// Modulation continue d'une centrale de brûleurs (T1, T2, C1 ou C2) :
// PID de température -> vitesse du ventilateur d'air de combustion
// (variateur) ; le gaz suit l'air par le régulateur de rapport air/gaz.
// Anticipation de la poussée : allure figée pendant la poussée, intégrale
// gelée, reprise sans à-coup puis supplément décroissant (bonus).
// Projet définitif V1 du 19/10/2026. Valeurs de départ, à recaler par l'essai E20.
// Une instance par centrale : DB115 (T1), DB116 (T2),
// DB117 (C1 = Zone Feu 1), DB118 (C2 = Zone Feu 2).
// Appel : OB35 (500 ms conseillé, dt = 0.5).
// La sécurité de flamme reste au coffret : ce bloc ne commande que l'allure.
// ====================================================================
FUNCTION_BLOCK FB115
VAR_INPUT
  dt        : REAL := 0.5;    // s, période d'appel
  PV        : REAL;           // °C, thermocouple K de la centrale
  PV_ok     : BOOL;           // thermocouple valide (pas de rupture)
  SP        : REAL;           // °C, consigne (recette / FB111)
  Pousse    : BOOL;           // pousseur en mouvement OU porte d'entrée ouverte (signal brut)
  Flamme_ok : BOOL;           // coffret : centrale en service, flammes présentes
  Auto      : BOOL;           // FAUX = manuel (U_man)
  U_man     : REAL := 60.0;   // %, allure en manuel
  Kp        : REAL := 2.0;    // % d'allure par °C d'écart
  Ti        : REAL := 180.0;  // s
  Td        : REAL := 20.0;   // s (0 = pas de dérivée)
  U_min     : REAL := 30.0;   // %, allure mini admise par CMI (stabilité de flamme)
  U_pousse  : REAL := 30.0;   // %, allure pendant la poussée (= l'actuelle « min »)
  Bonus     : REAL := 0.0;    // %, supplément après poussée (T1 20, C1 10, T2 0, C2 0)
  T_bonus   : REAL := 240.0;  // s, durée de décroissance du supplément
  T_filtre  : REAL := 2.0;    // s, anti-rebond des fins de course (parasites)
  V_max     : REAL := 5.0;    // %/s, variation maxi de l'allure
  F_min     : REAL := 25.0;   // Hz du ventilateur à 0 % (à fixer avec CMI)
  F_max     : REAL := 50.0;   // Hz à 100 %
END_VAR
VAR_OUTPUT
  U         : REAL;           // %, allure de la centrale
  F_Hz      : REAL;           // Hz, consigne du variateur d'air de combustion
  Etat      : INT;            // 0 régulation, 1 poussée, 2 reprise avec bonus, 3 manuel, 4 défaut
  Pousse_f  : BOOL;           // poussée filtrée
  Ecart     : REAL;           // °C, SP - PV
  Def_TC    : BOOL;           // thermocouple en défaut (allure maintenue)
END_VAR
VAR
  init       : BOOL;
  t_on       : REAL;
  t_off      : REAL;
  Pousse_old : BOOL;
  I_term     : REAL;
  D_f        : REAL;
  PV_old     : REAL;
  U_hold     : REAL;
  t_ap       : REAL;
END_VAR
VAR_TEMP
  e      : REAL;
  P_term : REAL;
  D_raw  : REAL;
  B_term : REAL;
  U_cmd  : REAL;
  dU     : REAL;
END_VAR

// ---------- 0. Initialisation (reprise sans à-coup sur l'allure manuelle)
IF NOT init THEN
  PV_old := PV; U := U_man; I_term := U_man; D_f := 0.0;
  t_ap := T_bonus; Pousse_old := FALSE; init := TRUE;
END_IF;

// ---------- 1. Anti-rebond du signal de poussée (parasites sur les fins de course)
IF Pousse THEN t_on := t_on + dt; t_off := 0.0; ELSE t_off := t_off + dt; t_on := 0.0; END_IF;
IF t_on >= T_filtre THEN Pousse_f := TRUE; ELSIF t_off >= T_filtre THEN Pousse_f := FALSE; END_IF;

// ---------- 2. Termes du PID (dérivée sur la mesure, filtrée)
e := SP - PV; Ecart := e;
P_term := Kp * e;
IF Td > 0.0 THEN
  D_raw := -Kp * Td * (PV - PV_old) / dt;
  D_f := D_f + dt / (Td / 8.0 + dt) * (D_raw - D_f);
ELSE
  D_f := 0.0;
END_IF;
PV_old := PV;

// ---------- 3. Fronts de poussée
IF Pousse_f AND NOT Pousse_old THEN
  U_hold := U;                                  // allure juste avant la poussée
END_IF;
IF (NOT Pousse_f) AND Pousse_old THEN
  t_ap := 0.0;                                  // début de la reprise
  I_term := U_hold - P_term - D_f;              // reprise sans à-coup depuis U_hold
END_IF;
Pousse_old := Pousse_f;
IF t_ap < T_bonus THEN
  B_term := Bonus * (1.0 - t_ap / T_bonus);
  t_ap := t_ap + dt;
ELSE
  B_term := 0.0;
END_IF;
```

```
// ---------- 4. Choix de l'allure
Def_TC := NOT PV_ok;
IF NOT Flamme_ok THEN
  Etat := 4; U_cmd := U_min; I_term := U_min;
ELSIF NOT PV_ok THEN
  Etat := 4; U_cmd := U;                        // thermocouple HS : allure maintenue, alarme
ELSIF NOT Auto THEN
  Etat := 3; U_cmd := U_man; I_term := U_man - P_term - D_f;
ELSIF Pousse_f THEN
  Etat := 1; U_cmd := U_pousse;                 // intégrale gelée pendant la poussée
ELSE
  IF B_term > 0.0 THEN Etat := 2; ELSE Etat := 0; END_IF;
  U_cmd := P_term + I_term + D_f + B_term;
  // anti-emballement : on n'intègre pas vers la butée
  IF NOT (((U_cmd >= 100.0) AND (e > 0.0)) OR ((U_cmd <= U_min) AND (e < 0.0))) THEN
    I_term := I_term + Kp * dt / Ti * e;
  END_IF;
  IF I_term > 100.0 THEN I_term := 100.0; ELSIF I_term < 0.0 THEN I_term := 0.0; END_IF;
  U_cmd := P_term + I_term + D_f + B_term;
END_IF;
IF U_cmd > 100.0 THEN U_cmd := 100.0; ELSIF U_cmd < U_min THEN U_cmd := U_min; END_IF;

// ---------- 5. Vitesse de variation et conversion en fréquence
dU := U_cmd - U;
IF dU > V_max * dt THEN dU := V_max * dt; ELSIF dU < -V_max * dt THEN dU := -V_max * dt; END_IF;
U := U + dU;
F_Hz := F_min + U / 100.0 * (F_max - F_min);
END_FUNCTION_BLOCK
```

FB117 COMPRESSEUR_VAR (S7-300/400) — compresseur à pression constante

```
// ================================================================
// FB117 COMPRESSEUR_VAR - Air comprimé - STEP 7 SCL (S7-300 ou S7-400)
// Pression constante par variateur sur le compresseur principal (appoint)
// + séquence du compresseur de base (charge / décharge).
// Projet définitif V1 du 19/10/2026. À utiliser SI le compresseur est raccordé à un automate ;
// sinon, mêmes réglages dans le PID interne du variateur (dossier,
// section compresseur) et séquence par le régulateur du compresseur de
// base. Valeurs de départ, à recaler par l'essai E21.
// Appel : OB35 (100 ms, dt = 0.1). Instance DB117.
// Les sécurités du compresseur (température d'huile, surpression,
// défaut moteur) restent dans son propre régulateur.
// ================================================================
FUNCTION_BLOCK FB117
VAR_INPUT
  dt        : REAL := 0.1;    // s
  P_bar     : REAL;           // bar, pression ballon principal (après sécheur)
  P_ok      : BOOL;
  Var_pret  : BOOL;           // variateur prêt, compresseur sans défaut
  Auto      : BOOL;
  SP        : REAL := 6.5;    // bar, consigne (besoin le plus exigeant + 0,5 bar)
  Kp        : REAL := 30.0;   // % de vitesse par bar d'écart
  Ti        : REAL := 15.0;   // s
  N_min     : REAL := 55.0;   // %, vitesse mini admise par le fabricant (50-60 %)
  N_man     : REAL := 80.0;   // %, vitesse en manuel
  dP_veille : REAL := 0.3;    // bar au-dessus de SP pour la veille
  t_veille  : REAL := 60.0;   // s à vitesse mini avant la veille
  dP_reprise: REAL := 0.2;    // bar sous SP pour la reprise
  Base_dispo: BOOL;           // compresseur de base disponible (si deux machines)
END_VAR
VAR_OUTPUT
  N_pct     : REAL;           // %, consigne de vitesse au variateur (100 % = 50 Hz)
  Marche    : BOOL;           // commande en charge du compresseur à variateur
  Base_ch   : BOOL;           // commande de charge du compresseur de base
  Etat      : INT;            // 0 régulation, 1 veille, 2 manuel, 3 défaut
END_VAR
VAR
  init    : BOOL;
  I_term  : REAL;
  t_min   : REAL;
  veille  : BOOL;
  t_haut  : REAL;
  t_bas   : REAL;
END_VAR
VAR_TEMP
  e     : REAL;
  N_cmd : REAL;
END_VAR

IF NOT init THEN I_term := N_man; init := TRUE; END_IF;
e := SP - P_bar;                                  // e > 0 : pression basse -> plus vite

IF NOT (P_ok AND Var_pret) THEN
  Etat := 3; Marche := FALSE; N_cmd := N_min; I_term := N_min;
ELSIF NOT Auto THEN
  Etat := 2; Marche := TRUE; N_cmd := N_man; I_term := N_man;
ELSE
  // ---- veille : vitesse mini et pression haute pendant t_veille
  IF (N_pct <= N_min + 0.5) AND (P_bar > SP + dP_veille) THEN
    t_min := t_min + dt;
  ELSE
    t_min := 0.0;
  END_IF;
  IF t_min >= t_veille THEN veille := TRUE; END_IF;
  IF veille AND (P_bar < SP - dP_reprise) THEN veille := FALSE; I_term := N_min; END_IF;
  IF veille THEN
    Etat := 1; Marche := FALSE; N_cmd := N_min;  // décharge / arrêt selon le régulateur machine
  ELSE
    Etat := 0; Marche := TRUE;
    N_cmd := Kp * e + I_term;
    IF NOT (((N_cmd >= 100.0) AND (e > 0.0)) OR ((N_cmd <= N_min) AND (e < 0.0))) THEN
      I_term := I_term + Kp * dt / Ti * e;
    END_IF;
    IF I_term > 100.0 THEN I_term := 100.0; ELSIF I_term < N_min THEN I_term := N_min; END_IF;
    N_cmd := Kp * e + I_term;
  END_IF;
END_IF;
IF N_cmd > 100.0 THEN N_cmd := 100.0; ELSIF N_cmd < N_min THEN N_cmd := N_min; END_IF;
N_pct := N_cmd;

// ---- séquence du compresseur de base (seulement s'il y a deux machines)
IF Base_dispo AND Auto THEN
  IF (N_pct >= 95.0) AND (P_bar < SP - 0.2) THEN t_haut := t_haut + dt; ELSE t_haut := 0.0; END_IF;
  IF N_pct <= 60.0 THEN t_bas := t_bas + dt; ELSE t_bas := 0.0; END_IF;
  IF t_haut >= 30.0 THEN Base_ch := TRUE; END_IF;
  IF t_bas >= 120.0 THEN Base_ch := FALSE; END_IF;
ELSE
  Base_ch := FALSE;
END_IF;
END_FUNCTION_BLOCK
```